

DEPARTMENT OF COMPUTER SCIENCE &
ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON

DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SUMMER 2026



AgriHome project logo: leaf and camera lens motif for vision-based agricultural monitoring.

TEAM AGRIHOME

GOVINDA AWASTHI
LAXMAN BHUSHAL
THUC DANG
SAGAR KHADKA
MUHAMMAD SAAD MEMON
ALEX TABLIZO
NATHAN NGUYEN TRAN

Revision History

Revision	Date	Author(s)	Description
0.1	06.16.2026	SK, TD	Initial DDS and hardware specification draft
0.2	06.17.2026	MSM, AT	Layer-level subsystem designs
0.3	06.18.2026	LB, NNT	Template formatting, logo, lists
1.0	06.18.2026	GA	Reviewed submission version
2.0	07.23.2026	Team	Reflect real Next.js/Postgres/Pi+Moonraker stack
2.1	07.28.2026	Team	Klipper edge: fswebcam primary; klipper_url; updateKlipperUrl

Contents

1	AH-109: Introduction and System Overview	6
1.1	Introduction	6
1.2	System Overview	6
2	AH-110: Frontend Layer Subsystems	7
2.1	Layer Purpose	7
2.2	Layer Hardware	7
2.3	Layer Operating System	7
2.4	Layer Software Dependencies	7
2.5	Dashboard Overview Subsystem	8
2.6	Image Feed Display Subsystem	8
2.7	Plant Health Result UI Subsystem	8
2.8	Monitoring History Subsystem	8
2.9	Alerts and Notifications UI Subsystem	8
2.10	Responsive Navigation and Layout Subsystem	8
3	AH-111: Backend API Layer Subsystems	9
3.1	Layer Purpose	9
3.2	Layer Hardware	9
3.3	Layer Operating System	9
3.4	Layer Software Dependencies	9
3.5	Authentication and User Management API	9
3.6	Plant and Tray Management API	9
3.7	Image Upload and Retrieval API	9
3.8	Edge Device API (replaces draft “sensor data API”)	10
3.9	Edge Service Mapping (implementation anchors)	10
3.10	Plant Health Analysis API	10
3.11	Alerts API	10
4	AH-112: Database / Data Layer Subsystems	11
4.1	Layer Purpose	11
4.2	Layer Hardware	11
4.3	Layer Operating System	11
4.4	Layer Software Dependencies	11
4.5	Database Schema Summary	11
4.6	tray_systems Table	12
4.7	plants Table	12
4.8	camera_captures Table	12
4.9	edge_devices Table	12
4.10	edge_device_commands Table	12
4.11	capture_pose_sequences / capture_poses	12
4.12	capture_schedules Table	12
4.13	prediction_results / plant_reports / monitoring_events	12
4.14	Indexing, Retention, and Validation Strategy	12

5	AH-113: Storage File Management Systems	13
5.1	Layer Purpose	13
5.2	Layer Hardware	13
5.3	Layer Operating System	13
5.4	Layer Software Dependencies	13
5.5	File Management Subsystems	13
5.6	Image Upload Subsystem	13
5.7	File Naming and Folder Organization Subsystem	13
5.8	Image Retrieval Subsystem	13
5.9	Image Cleanup and Archiving Subsystem	14
5.10	Security and Access Control Subsystem	14
6	AH-114: Hardware / Camera Layer Subsystems	14
6.1	Layer Purpose	14
6.2	Layer Hardware	14
6.3	Layer Operating System	14
6.4	Layer Software Dependencies	14
6.5	Camera Capture Subsystem	14
6.6	Plant Rack Positioning Subsystem	15
6.7	Device Communication Subsystem	15
6.8	Power and Reliability Subsystem	15
6.9	Hardware Layer Limitations	15
7	AH-115: System Overview and Architecture Diagram	15
7.1	Architecture Explanation	15
7.2	Architecture Diagram	15
7.3	Architecture Data Flow	16
7.4	Interface Summary	16
7.5	Design Rationale	17
7.6	Current Implementation Connection	17
8	AH-116: Final Review and Submit	17
8.1	Final Review Checklist	17
8.2	Revision History Table	17
8.3	List of Figures / Tables	17
8.4	Appendix Suggestions	17
9	Appendix A: Suggested Supporting Material	17
10	Appendix B: Subsystem → Module / Table Map	18

List of Figures

1	AgriHome runtime architecture (AH-109 overview).	7
2	UI ↔ Next API ↔ Postgres/files; Pi agent ↔ register/heartbeat/ingest; optional CV.	15

List of Tables

1	AgriHome system layers.	6
2	Frontend layer software dependencies.	7
3	Dashboard Overview subsystem design details.	8
4	Image Feed Display subsystem design details.	8
5	Edge capabilities mapped to real modules and tables.	10
6	Actual PostgreSQL tables used by AgriHome.	11
7	Storage subsystem summary.	13
8	Hardware prototype component summary.	14
9	Backend API endpoint summary (selected real interfaces).	16
10	DDS subsystems mapped to real modules and tables.	18

1 AH-109: Introduction and System Overview

1.1 Introduction

This Detailed Design Specification describes the internal design of **AgriHome** as implemented for CSE 4317 Summer 2026. It expands requirements and architecture into layer-level and subsystem-level designs for the Vision Console, Next.js API routes, PostgreSQL data model, local file storage, and Raspberry Pi / Agri-Home/klipper capture path.

Earlier draft wording hypothesized a separate Express backend and fictional tables such as `users`, `images`, `sensor_readings`, and `health_results`. **This revision documents only the live stack:**

- **Frontend:** Next.js App Router Vision Console with AppShell (Overview, Trays, Add Plant, Mesh, Feedback, Schedule, **Devices**, Settings).
- **Backend:** Next.js Route Handlers under `src/app/api/**` (not Express), including `/api/raspberry-pi/*`, `/api/devices/*`, trays, plants, camera, schedules, files, health, and GraphQL.
- **Database:** PostgreSQL tables `tray_systems`, `plants`, `camera_captures`, `edge_devices`, `edge_device_commands`, `capture_pose_sequences`, `capture_poses`, `capture_schedules`, `prediction_results`, `plant_reports`, `monitoring_events` (plus mesh/feedback helpers). Base domain tables and edge tables are in `db/schema.sql`; migrations 009–012 upgrade existing DBs (`011_capture_poses.sql` adds pose sequence tables; `012_klipper_url.sql` renames `moonraker_url` → `klipper_url`).
- **Storage:** `STORAGE_*` originals/processed/temp on the app host, served by GET `/api/files/...`
- **Hardware:** Raspberry Pi + Agri-Home/klipper (`fswebcam` via `camera-macros/save_image.sh`), optional LAN HTTP streamer for server-side stills, and the `agrihome_agent` Python package; hinge/motor pose fields are modeled (motion may be stubbed).
- **Optional CV:** cv-backend via `CV_SPECIES_INFERENCE_URL`.

1.2 System Overview

Operators use the Vision Console. Edge agents register and heartbeat against AgriHome. Capture is primarily queued (agent runs `fswebcam / save_image.sh` and multipart-ingests); optional immediate path pulls an HTTP streamer still when `klipper_url` is set. Domain truth lives in PostgreSQL; binaries live on disk. Optional inference writes predictions and reports.

Layer	Role
Frontend (AH-110)	Vision Console UI for trays, plants, devices, schedules
Backend API (AH-111)	Next.js routes + edge/domain services
Database (AH-112)	PostgreSQL structured records
Storage (AH-113)	Local image files linked by <code>camera_captures.image_url</code>
Hardware (AH-114)	Pi agent, Klipper/fswebcam, optional pose rig

Table 1: AgriHome system layers.

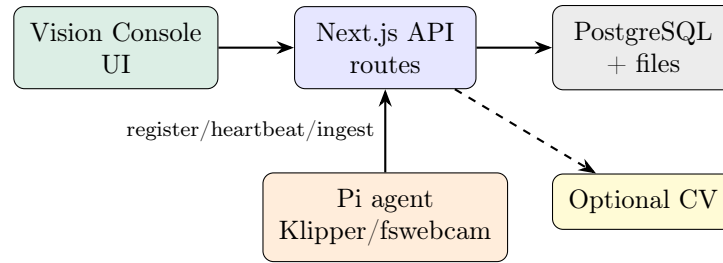


Figure 1: AgriHome runtime architecture (AH-109 overview).

2 AH-110: Frontend Layer Subsystems

2.1 Layer Purpose

Present authenticated operator workflows for monitoring trays and plants, managing schedules and meshes, viewing captures, and controlling edge devices (Devices page and tray Raspberry Pi panel Take Picture).

2.2 Layer Hardware

Any modern laptop, desktop, tablet, or phone with a contemporary browser. No client GPU is required.

2.3 Layer Operating System

OS-agnostic browser runtime (macOS, Windows, Linux, iOS/Android browsers). PWA install is supported where the browser allows.

2.4 Layer Software Dependencies

Dependency	Role
Next.js App Router / React	Dashboard routes and components
TypeScript + Tailwind CSS	Typed UI and styling
Firebase client SDK	Operator sign-in
Recharts	Health / monitoring charts
AppShell + design atoms	Nav shell, FAB, status badges

Table 2: Frontend layer software dependencies.

2.5 Dashboard Overview Subsystem

Aspect	Design
Route	<code>/dashboard</code> (protected)
Purpose	Summarize trays, health, latest frame, and monitoring status
Data	Tray list/health snapshot; latest camera frame when available
Modules	Server components / tray topology reads; auth gate in protected layout
Failure	Empty states when no trays; session redirect when unauthenticated

Table 3: Dashboard Overview subsystem design details.

2.6 Image Feed Display Subsystem

Shows latest tray/plant imagery from `camera_captures` via `/api/files/...` URLs. Tray detail embeds camera panels; plant detail supports photo upload and last-image display through plant media helpers (`PlantImage`).

Aspect	Design
Purpose	Visual record of current plant / tray condition
Source table	<code>camera_captures</code> (<code>image_url</code> , <code>captured_at</code> , pose fields)
APIs	GET <code>/api/camera/latest</code> , file bytes via GET <code>/api/files/...</code>
UI	Loading/error states; refresh for newest metadata

Table 4: Image Feed Display subsystem design details.

2.7 Plant Health Result UI Subsystem

Plant and tray pages surface health scores/status, latest diagnosis text, prediction/report history, and charts. Photo-first add-plant flows call species inference when `CV_SPECIES_INFERENCE_URL` is configured. Data comes from `plants`, `prediction_results`, and `plant_reports`.

2.8 Monitoring History Subsystem

Monitoring events and reports are listed per tray/plant. Historical captures remain browsable subject to disk retention. Schedule pages show next/last run metadata from `capture_schedules`.

2.9 Alerts and Notifications UI Subsystem

In-console alerts appear as `monitoring_events` and status badges (healthy/warning/critical). Device stale marking surfaces offline Pis on the Devices page. Remote push/email remains future scope.

2.10 Responsive Navigation and Layout Subsystem

`AppShell` provides desktop sidebar and mobile bottom navigation with FAB for Add Plant. Primary nav: Overview, Trays, Mesh, Feedback (preference-gated), Schedule, and **Devices**. Settings is

appended. Safe-area padding supports mobile PWA use.

TrayEdgeDevicePanel (tray detail) exposes Take Picture, Klipper/streamer URL edit, device link, and pose-generation entry points for provisioned Pis.

3 AH-111: Backend API Layer Subsystems

3.1 Layer Purpose

Authenticate operators and devices, orchestrate domain services, persist to PostgreSQL/files, queue edge commands, and optionally call CV inference.

3.2 Layer Hardware

Application host (developer laptop, lab server, or container) with network access to PostgreSQL and, for optional server-side stills, to Pi HTTP streamer URLs stored as `klipper_url`.

3.3 Layer Operating System

Linux or macOS hosts are typical; Node.js runtime as pinned by the repository.

3.4 Layer Software Dependencies

Next.js Route Handlers, pg pool, Firebase Admin/session verification, multipart parsers for ingest, and HTTP clients for optional streamer stills + CV URLs.

3.5 Authentication and User Management API

- `POST /api/auth/session` — establish server session from Firebase ID token.
- `POST /api/auth/logout` — clear session cookie.
- Protected handlers require verified session email for ownership scoping (`owner_email`).

Device identity is separate: header `X-Agrihome-Device-Key` on `/api/raspberry-pi/*` routes (hashed key in `edge_devices`).

3.6 Plant and Tray Management API

Representative endpoints: `GET/POST /api/trays`, `GET/PATCH /api/trays/{trayId}`, `POST /api/trays/{trayId}`, `GET/POST /api/plants`, `POST /api/plants/from-photo`, `POST /api/plants/manual`, `GET/PATCH/DELETE /api/plants/{plantId}`, photo upload, reports, mesh, schedules, monitoring log, preferences, and GraphQL.

3.7 Image Upload and Retrieval API

- Operator/camera: `POST /api/camera/ingest`, `GET /api/camera/latest`.
- Edge: `POST /api/raspberry-pi/ingest` (multipart; device key).
- Retrieval: `GET /api/files/[...path]` for stored originals/processed.

3.8 Edge Device API (replaces draft “sensor data API”)

AgriHome does not expose a generic environmental-sensor REST surface in the Summer 2026 cut. Edge capture is first-class:

- POST `/api/raspberry-pi/register` — provisioning code + fingerprint → device, one-time API key, linked tray.
- POST `/api/raspberry-pi/heartbeat` — online status + command claim.
- GET `/api/raspberry-pi/commands`, `/poses`, `/capture-plan`.
- GET `/api/devices`, GET/POST `/api/devices/{deviceId}` actions: `capture`, `linkTray`, `revoke`, `rotateKey`, `updateKlipperUrl` (alias `updateMoonrakerUrl`), `updateLimits`.
- GET/POST `/api/trays/{trayId}/poses` — pose sequences for multi-angle capture.

3.9 Edge Service Mapping (implementation anchors)

Capability	Service module	Primary tables / APIs
Edge registration	<code>edge-device-service.ts</code>	<code>edge_devices</code> , <code>tray_systems</code> ; POST <code>.../register</code>
Heartbeat / stale	<code>edge-device-service.ts</code>	<code>edge_devices.last_heartbeat_at</code> ; POST <code>.../heartbeat</code>
Capture commands	<code>edge-command-service.ts</code>	<code>edge_device_commands</code> ; claim on heartbeat
Ingest storage	<code>edge-capture-service.ts</code> , <code>camera/storage helpers</code>	<code>files</code> + <code>camera_captures</code> ; POST <code>.../ingest</code>
Plant attach	<code>edge-plant-attach-service.ts</code>	<code>plants</code> , pose slot hints
Disease hook	<code>edge-disease-hook.ts</code>	<code>prediction_results</code> / <code>plant_reports</code> , <code>monitoring_events</code>
Vision hook	<code>edge-vision-hook.ts</code>	optional auto vision after in- gest
Postprocess	<code>edge-capture-postprocess.ts</code>	attach + optional hooks after save

Table 5: Edge capabilities mapped to real modules and tables.

3.10 Plant Health Analysis API

Predictions via GET `/api/predictions/latest` and service-layer calls to `CV_SPECIES_INFERENCE_URL` / tray vision. Optional auto vision after Pi ingest when `DEVICE_AUTO_VISION_ON_INGEST` is enabled.

3.11 Alerts API

Monitoring events are created/read through monitoring services and `/api/monitoring/log`. Device stale marking runs when listing devices. Dedicated push notification API remains future.

4 AH-112: Database / Data Layer Subsystems

4.1 Layer Purpose

Provide durable relational storage for AgriHome domain entities and edge lifecycle state.

4.2 Layer Hardware

PostgreSQL server (local Docker, lab VM, or managed instance).

4.3 Layer Operating System

PostgreSQL-supported OS; accessed over TCP from the Next.js host.

4.4 Layer Software Dependencies

PostgreSQL 14+ recommended; SQL under `db/schema.sql` and `db/migrations/` (`009_edge_devices.sql`, `010_camera_ingest_metadata.sql`, `011_capture_poses.sql`).

4.5 Database Schema Summary

The live schema does **not** use fictional `users`, `images`, `sensor_readings`, or `health_results` tables. Firebase identifies humans; PostgreSQL stores domain rows keyed largely by `owner_email` and string IDs.

Table	Migration	Role
<code>tray_systems</code>	base / 009	Tray metadata, health, optional <code>edge_device_id</code>
<code>plants</code>	base	Plants in trays; slot/row/column; diagnosis fields
<code>camera_captures</code>	base / 010	Capture metadata + image URL + pose fields
<code>edge_devices</code>	009+012	Pi identity, key hash, <code>klipper_url</code> , heart-beat
<code>edge_device_commands</code>	009	Pending/claimed/completed device commands
<code>capture_pose_sequences</code>	011	Active pose sequences per tray
<code>capture_poses</code>	011	Ordered hinge/motor/dwell poses
<code>capture_schedules</code>	base	Interval schedules; destination may be <code>raspberrypi-edge</code>
<code>prediction_results</code>	base	Classifier outputs linked to captures
<code>plant_reports</code>	base	Structured diagnosis reports
<code>monitoring_events</code>	base	Alert/log stream
<code>mesh_networks</code>	base	Tray groupings
<code>feedback_ingest</code> / <code>user_preferences</code>	base	ML feedback opt-in & samples

Table 6: Actual PostgreSQL tables used by AgriHome.

4.6 tray_systems Table

Primary key `id`; `owner_email`, `name`, `zone`, `crop`, `counts`, `health/status`, `device_id`, nullable `edge_device_id`, `last_capture_at`, `timestamps/vision summary` fields.

4.7 plants Table

Primary key `id`; `tray_id` FK; `mesh IDs` JSON; `name/cultivar/slot/indices`; `health`; `latest_diagnosis`; optional `description`; `last_image_url/last_image_at`.

4.8 camera_captures Table

Replaces fictional images. Stores `image_url`, `captured_at`, `source`, `status`, optional `plant_id`, `hinge_deg`, `motor_mm`, `pose_order`, `command_id` (migration 010).

4.9 edge_devices Table

`cpu_serial` unique; `api_key_hash + api_key_prefix`; `klipper_url`; `status`; `last_heartbeat_at`; `hinge/motor range` fields; `revoked_at`.

4.10 edge_device_commands Table

Command queue: `command_type` (e.g. `capture_now`, `sync_poses`, `reboot_agent`), `payload_json`, `status`, `claim/complete` timestamps, optional `result/error`.

4.11 capture_pose_sequences / capture_poses

Multi-angle capture plans for trays/devices (AGRI-104). Poses include `hinge degrees`, `motor mm`, `dwel`, optional `plant/slot binding`.

4.12 capture_schedules Table

`interval_minutes`, `next_run_at`, `destination` defaulting to `computer-vision-backend` or set to `raspberrypi-edge`.

4.13 prediction_results / plant_reports / monitoring_events

Persist CV outputs, structured reports (`diseases/deficiencies JSON`), and operator-visible events. These supersede earlier draft `health_results/alerts/monitoring_history` names.

4.14 Indexing, Retention, and Validation Strategy

Indexes exist on owner emails, edge status, pending commands, tray/plant captures, and pose ordering. Application services validate ownership and foreign keys. Image retention is operational (disk capacity); no automatic cold-archive job is mandated in the prototype.

5 AH-113: Storage File Management Systems

5.1 Layer Purpose

Persist binary imagery referenced by relational rows without forcing blobs into git or a single cloud vendor during development. Files live on the **application host** (not on a per-user Pi SSD as an earlier draft suggested).

5.2 Layer Hardware

Local disk or mounted volume on the Next.js host.

5.3 Layer Operating System

Host filesystem semantics (POSIX typical).

5.4 Layer Software Dependencies

STORAGE_ROOT, STORAGE_ORIGINALS_DIR, STORAGE_PROCESSED_DIR, STORAGE_TEMP_DIR resolution helpers; Next.js file route; `save-original` / leaf storage helpers used by edge ingest.

5.5 File Management Subsystems

Subsystem	Processing
Image Upload	Validate type/size; write original; insert <code>camera_captures</code>
Folder organization	Generated names under originals/processed/temp trees
Image Retrieval	GET <code>/api/files/[...path]</code> streams allowlisted bytes
Cleanup / Archiving	Manual/ops for prototype; future cold archive optional
Security	Session for operator uploads; device key for Pi ingest

Table 7: Storage subsystem summary.

5.6 Image Upload Subsystem

Accepts operator uploads and device multipart ingest; writes files then inserts `camera_captures` with a stable `/api/files/...` URL.

5.7 File Naming and Folder Organization Subsystem

Opaque/generated filenames under logical folders as implemented by storage helpers; keys stored in DB rather than user-supplied paths.

5.8 Image Retrieval Subsystem

GET `/api/files/[...path]` streams bytes for Console `next/image` / media components with local pattern allowlisting.

5.9 Image Cleanup and Archiving Subsystem

Manual/ops cleanup for the prototype; future archive jobs may move cold originals off primary disk.

5.10 Security and Access Control Subsystem

File routes are application-mediated. Device ingest requires device API key. Operator uploads require session auth. Host OS permissions must protect `STORAGE_*` directories.

6 AH-114: Hardware / Camera Layer Subsystems

6.1 Layer Purpose

Capture tray imagery on the bench and deliver it to AgriHome with minimal operator toil.

6.2 Layer Hardware

- Raspberry Pi (or compatible) running Agri-Home/klipper with `fswebcam`.
- Primary stills via `camera-macros/save_image.sh`; optional HTTP streamer stills (e.g. `/webcam/?action=snap` on nginx :80) when `klipper_url` is configured for server-side Take Picture.
- Optional hinge/motor rig for multi-pose capture (fields modeled; motion may be stubbed).
- Plant tray/rack fixture and lighting as built in makerspaces.

Component	Role
Raspberry Pi	Runs <code>agrihome_agent</code> + Agri-Home/klipper
Klipper / fswebcam	Primary JPEG source via <code>save_image.sh</code>
AgriHome agent	Register, heartbeat, claim commands, multipart ingest
Vision Console	Operator Take Picture / Devices / Klipper URL edit

Table 8: Hardware prototype component summary.

6.3 Layer Operating System

Raspberry Pi OS (or similar Linux) with Python 3 for `agrihome_agent`.

6.4 Layer Software Dependencies

Agri-Home/klipper, `fswebcam`, `agrihome_agent` (register/run) from the Agri-Home/moonraker companion repo (`feature/agrihome-bridge`; configured for `klipperUrl` + `fswebcam`), network access to AgriHome base URL, shared `DEVICE_PROVISIONING_SECRET`. Ops detail: `docs/ops/RASPBERRY_PI_KLIPPER.md`.

6.5 Camera Capture Subsystem

Two paths:

1. **Primary — agent + fswebcam:** command `capture_now` claimed on heartbeat; agent runs Agri-Home/klipper `camera-macros/save_image.sh` and POSTs multipart ingest.

2. **Optional — HTTP streamer still:** when `klipper_url` is set and reachable, AgriHome may fetch still candidates (common `:80` webcam paths) via `captureFromKlipperStreamerDirect` in `edge-capture-service.ts` before queuing the agent.

6.6 Plant Rack Positioning Subsystem

Pose sequences define hinge degrees and motor millimeters per plant slot. Agent walks poses when `runPoses` is set; failed poses should not abort the entire sequence. Physical motion drivers may be stubs pending hardware bring-up. Tables: `capture_pose_sequences`, `capture_poses`.

6.7 Device Communication Subsystem

Agent ↔ AgriHome over HTTP(S): register, heartbeat, commands, poses, capture-plan, ingest. Console ↔ AgriHome for device actions. AgriHome ↔ optional HTTP streamer for direct stills.

6.8 Power and Reliability Subsystem

USB/wall power for Pi and lighting; heartbeat stale window (`DEVICE_HEARTBEAT_STALE_MINUTES`) marks offline devices; revoke/re-provision recovers credentials after compromise or replacement.

6.9 Hardware Layer Limitations

- Server-side snapshot requires AgriHome host reachability to the Pi (LAN/WARP caveats).
- Optional streamer URL paths vary by nginx/mjpegstreamer configuration.
- No autonomous plant-care actuation (watering/dosing) in scope.
- Environmental sensors remain optional relative to webcam capture.

7 AH-115: System Overview and Architecture Diagram

7.1 Architecture Explanation

AgriHome is a layered web application with an edge capture extension. The UI never talks to PostgreSQL directly. Device credentials are not Firebase users. CV is an optional processing adapter.

7.2 Architecture Diagram

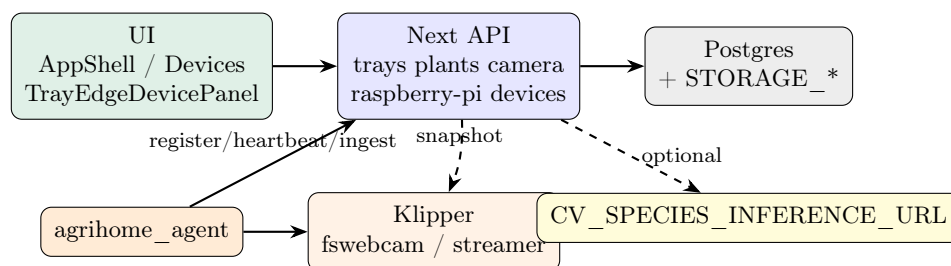


Figure 2: UI ↔ Next API ↔ Postgres/files; Pi agent ↔ register/heartbeat/ingest; optional CV.

Mermaid equivalent (documentation form):

flowchart LR

```

UI[Vision Console] --> API[Next.js API]
API --> DB[(PostgreSQL)]
API --> FS[STORAGE_* files]
Agent[agrihome_agent] -->|register/heartbeat/ingest| API
Agent --> MR[Klipper fswebcam]
API -->|optional snapshot| MR
API -->|CV_SPECIES_INFERENCE_URL| CV[cv-backend]

```

7.3 Architecture Data Flow

1. Agent registers → `edge_devices` + tray + one-time API key.
2. Heartbeat loop updates presence and claims commands from `edge_device_commands`.
3. Operator Take Picture → queued `capture_now` (fswebcam) or optional streamer still → file + `camera_captures` row; plant attach via `edge-plant-attach-service`.
4. Optional disease/vision hooks → `prediction_results` / `plant_reports` + `monitoring_events`.
5. Console reads trays/plants/devices/captures for display.

7.4 Interface Summary

Method	Path	Auth / Purpose
POST	/api/raspberry-pi/register	Provisioning secret; create device+tray
POST	/api/raspberry-pi/heartbeat	Device key; presence + claim commands
POST	/api/raspberry-pi/ingest	Device key; multipart image upload
GET	/api/raspberry-pi/commands	Device key; pending commands
GET	/api/raspberry-pi/poses	Device key; active pose sequence
GET	/api/raspberry-pi/capture-plan	Device key; schedules+poses+limits
GET	/api/devices	Session; list devices
POST	/api/devices/{id}	Session; capture/link/revoke/rotate/URL
GET/POST	/api/trays/{id}/poses	Session; configure poses
GET/POST	/api/trays, /api/plants	Session; tray/plant CRUD
GET	/api/files/...	App-mediated file bytes
GET	/api/health	DB + inference configuration probe

Table 9: Backend API endpoint summary (selected real interfaces).

7.5 Design Rationale

Reusing Agri-Home/klipper camera macros (`fswebcam`) avoids a custom camera daemon. Next.js Route Handlers keep one deployable unit for UI+API. Hashed device keys and a rotatable provisioning secret match the bench threat model without inventing a second human identity system in PostgreSQL.

7.6 Current Implementation Connection

Source anchors: `src/app/api/raspberry-pi/*`, `src/app/api/devices/*`, `src/lib/services/edge-*.ts`, `TrayEdgeDevicePanel.tsx`, `db/schema.sql`, migrations 009–012, and ops docs for Pi/Klipper and edge security.

8 AH-116: Final Review and Submit

8.1 Final Review Checklist

- ☐ Cover, roster, revision 2.1 (07.28.2026), Summer 2026 footer present.
- ☐ AH-109–AH-116 sections complete with real schema/endpoints.
- ☐ Frontend includes Devices nav and Take Picture panel behavior.
- ☐ Backend documented as Next.js API routes (not Express).
- ☐ Hardware section covers Pi + Klipper/fswebcam + agent + optional streamer.
- ☐ Interface table matches implemented raspberry-pi/devices routes.
- ☐ Edge services mapped: registration, heartbeat, commands, ingest, plant attach, disease hook.
- ☐ Cross-check with SRS requirements and Charter milestones.

8.2 Revision History Table

See front-matter revision history (includes 2.1 / 07.28.2026 Klipper edge update).

8.3 List of Figures / Tables

Generated automatically by `\listoffigures` and `\listoftables` in this document's front matter.

8.4 Appendix Suggestions

Attach agent README excerpts, example `agent.json`, sample register/ingest curls, and Optional streamer URL discovery notes and `save_image.sh` smoke-test output.

9 Appendix A: Suggested Supporting Material

- Ops: `docs/ops/RASPBERRY_PI_KLIPPER.md` (Moonraker stub superseded).
- Ops: Edge device security (AGRI-109) controls checklist.
- Implementation guide sections for storage and CV env vars.

- Physical E2E checklist: migrate, provision, heartbeat online, Take Picture, optional poses/schedules.

10 Appendix B: Subsystem → Module / Table Map

DDS subsystem	Code / routes	Tables
Dashboard / nav	AppShell, dashboard routes	tray_systems, monitoring_events
Image feed	camera UI, /api/files, /api/camera/*	camera_captures
Plant health UI	plant/tray pages, predictions APIs	plants, prediction_results, plant_reports
Edge registration	edge-device-service, /api/raspberry-pi/register	edge_devices, tray_systems
Heartbeat	updateDeviceHeartbeat, /api/raspberry-pi/heartbeat	edge_devices
Capture commands	edge-command-service, De- vices action=capture	edge_device_commands
Ingest / snapshot	edge-capture-service, /api/raspberry-pi/ingest	camera_captures + files
Plant attach	edge-plant-attach-service	plants, capture_poses
Disease hook	edge-disease-hook	prediction_results, plant_reports, monitoring_events
Poses	capture-pose-service, trays/poses APIs	capture_pose_sequences, capture_poses
Schedules	schedule APIs / capture-plan	capture_schedules

Table 10: DDS subsystems mapped to real modules and tables.

References

- [1] AgriHome, *System Requirements Specification*, updated Summer 2026.
- [2] AgriHome, *Architectural Design Specification*, CSE 4316, Spring 2026.
- [3] AgriHome, *Project Charter*, updated Summer 2026.
- [4] Vercel, Inc., “Next.js Documentation,” <https://nextjs.org/docs>.
- [5] PostgreSQL Global Development Group, “PostgreSQL Documentation,” <https://www.postgresql.org/docs/>.
- [6] Google, “Firebase Authentication,” <https://firebase.google.com/docs/auth>.
- [7] Agri-Home/klipper companion firmware and camera-macros/save_image.sh; AgriHome ops guide docs/ops/RASPBERRY_PI_KLIPPER.md.